Capacity, Sharding, and Python Integration

A public dataset becomes a database.
The next import arrives tomorrow.

Day 1: workload measurements and distribution choices
Day 2: a repeatable import with a checked result

A measurable workload

Measurement	Meaning	Example unit
Latency	Time for one request	milliseconds
Throughput	Completed requests per interval	requests per second
Working set	Data and index pages used repeatedly	bytes
Growth horizon	Time covered by a projection	days

A daily report and an interactive lookup can use the same records and still have different response-time requirements.

The average can hide a slow tail

```
from math import ceil
latencies_ms = [20] * 18 + [500, 1000]
ordered = sorted(latencies_ms)
mean_ms = sum(ordered) / len(ordered)
p95_ms = ordered[ceil(0.95 * len(ordered)) - 1]
print(mean_ms, p95_ms)
```

```
Mean: 93 ms     Nearest-rank p95: 500 ms
```

Synthetic 20-request trace. A small example of the calculation, not a measurement of Atlas, Supabase, or this public dataset.

Capacity options address different pressures

Option	Pressure it can address	What still needs a decision
Tune a query or index	Unnecessary examined work	Write cost and result correctness
Scale up	One node lacks CPU or memory	Cost and the next capacity limit
Cache or read replicas	Repeated or distributable reads	Acceptable staleness
Partition within a database	Access or retention by a key	Pruning and partition maintenance
Shard across nodes	Work or data needs distribution	Routing, balancing, and recovery

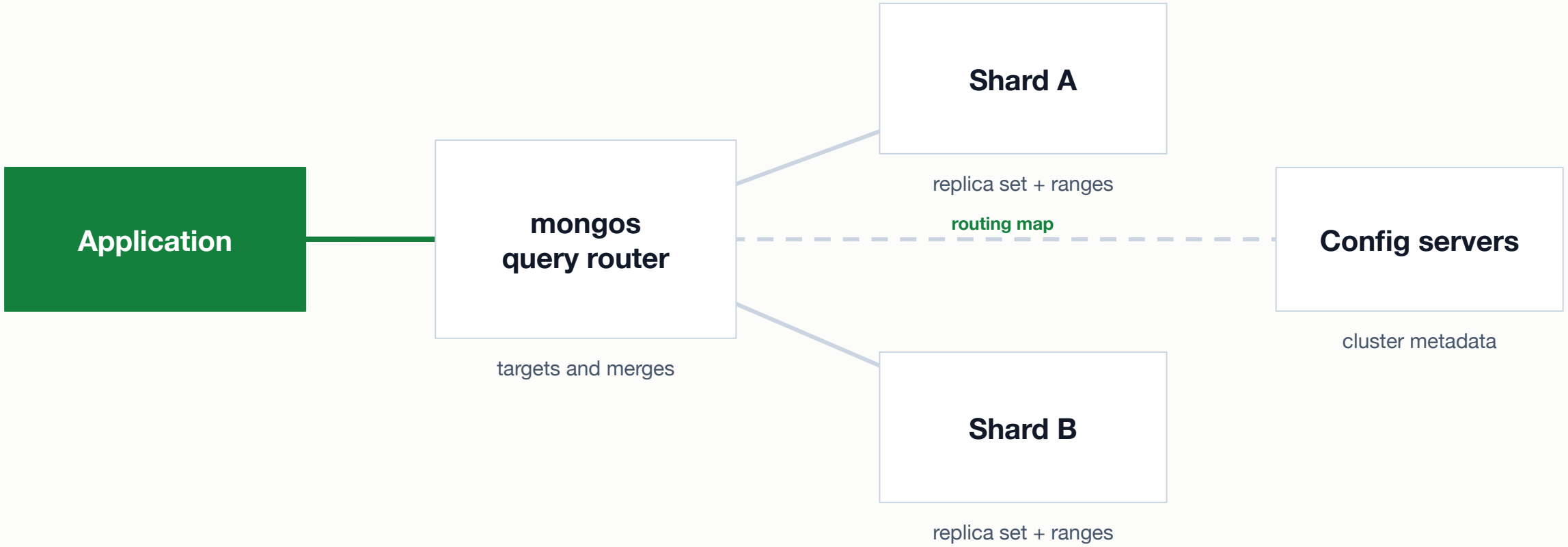
The measured workload determines which option is worth testing.

Replication and sharding

Placement of a 75-record collection	What each member holds
One replica set with three members	Each member maintains the same logical 75 records
Three shards with a balanced split	Each shard owns about 25 of the 75 records
Each shard also has replicas	Its replicas copy that shard's assigned portion

The shard split above is an illustration, not a guaranteed balance. Replication still matters after a dataset has been divided.

Inside a sharded MongoDB cluster



The shard key determines distribution and whether a query can target a limited shard set.

A versioned public source


```
{
  "cveID": "CVE-2026-56291",
  "vendorProject": "Balbooa",
  "product": "Forms",
  "dateAdded": "2026-07-10",
  "cwes": ["CWE-434"]
}
```


CISA KEV catalog
75 selected records
Catalog: July 10, 2026
Retrieved: July 13, 2026

Selected fields shown. dateAdded means catalog inclusion.
The historical subset is not a random sample or a current security assessment.

Checks before an import

Observation	Notebook behavior
Missing vendor, ID, or required text	Stop before loading
2026-02-30 or compact 20260201	Reject an invalid or noncanonical date
Repeated CVE identifier	Stop and resolve which record is authoritative
cwes is text instead of a list	Reject the unexpected representation
Vendor label "SimpleHelp "	Show the trailing space and preserve the source

Parsing JSON successfully does not establish that its fields fit the model.

Cardinality and frequency measure different things

Eight synthetic tickets	Distinct values	Largest value's share
Six open, two resolved	2 statuses	6 / 8 = 75%
A different ID on each ticket	8 IDs	1 / 8 = 12.5%

Cardinality counts distinct values. Frequency counts their repetition.
A unique ID distributes records finely, but one ID can still receive most reads.

Record frequency and request frequency require different measurements.

Measured keys in the 75-record snapshot

Candidate	Distinct values	Largest value count	Largest share
cveID	75	1	1.33%
dateAdded	38	8	10.67%
vendorProject	42	16	21.33%
vendor + product	56	4	5.33%

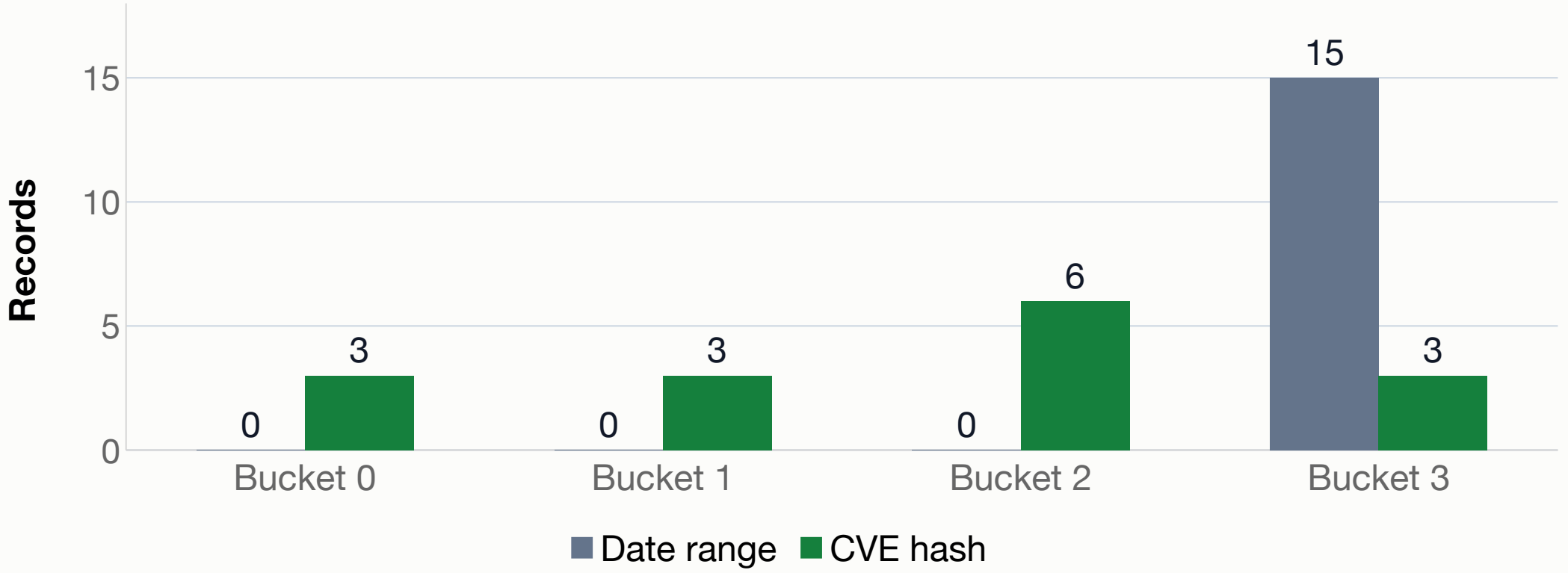
All four measurements describe this selected dataset.
They do not measure query traffic, latency, or a production bottleneck.

Ranged placement follows the key order

Teaching bucket	Date range
0	Before April 22
1	April 22 through May 19
2	May 20 through May 31
3	June 1 onward

The oldest 60 records establish these 2026 boundaries.
The newest 15 records model later writes into fixed ranges.
All 15 land in bucket 3.

The same later records under two placement rules



Fifteen later records, four fixed teaching buckets, SHA-256 modulo 4.
This omits MongoDB balancing, range migration, and request latency.

Hashing preserves equality

```
from hashlib import sha256
candidate = "Microsoft"
bucket = int(sha256(candidate.encode()).hexdigest(), 16) % 4
print(bucket)
```

Output: 1. All 16 identical "Microsoft" values map to that same bucket.

Hashing does not split repeated values into different keys.
The notebook hashes distinct CVE IDs, not vendor labels.

Query targeting depends on the key

Query with a hashed cveID shard key	Routing implication
Exact cveID equality	The key can identify the relevant shard
Only vendorProject = Microsoft	The filter lacks the shard key
Only dateAdded in a time window	The filter lacks the shard key
Count every record by vendor	The result needs data from across the collection

A local index may reduce work within each shard without narrowing routing.
A whole-catalog aggregate can legitimately need several shards.

A storage projection with explicit assumptions

Assumption	Value
Initial data retained	100 MiB
New data retained per day	20 MiB
Planning horizon	30 days
Assumed overhead multiplier	2

```
(100 + 20 x 30) x 2 = 1,400 MiB
```

The multiplier is an assumption, not a provider billing rule.
1 MiB = 1,048,576 bytes. 1 MB = 1,000,000 bytes.

Lab, Part 1: inspect and compare

Open Notebook 06 and use its embedded snapshot. Run Sections 1 and 2. Compare cveID with dateAdded using two displayed measurements.

Explain whether either candidate fits your query and whether this 75-record database needs sharding.

Save your notebook for Day 2. There is no separate Day 1 submission. Work individually. No paid cluster or sharded deployment is needed.

Day 2: Python Imports and Verification

The import ran yesterday.
Today it runs again.

A stable identifier makes a repeat possible.
A query and a value comparison tell us what was actually stored.

One database path is enough

TARGET	Where the database runs	Connection requirement
sqlite	Inside this notebook process	Python standard library
atlas	Your MongoDB Atlas project	PyMongo, database user, runtime IP rule
postgres	Your Supabase/PostgreSQL project	Psycopg and database connection URL

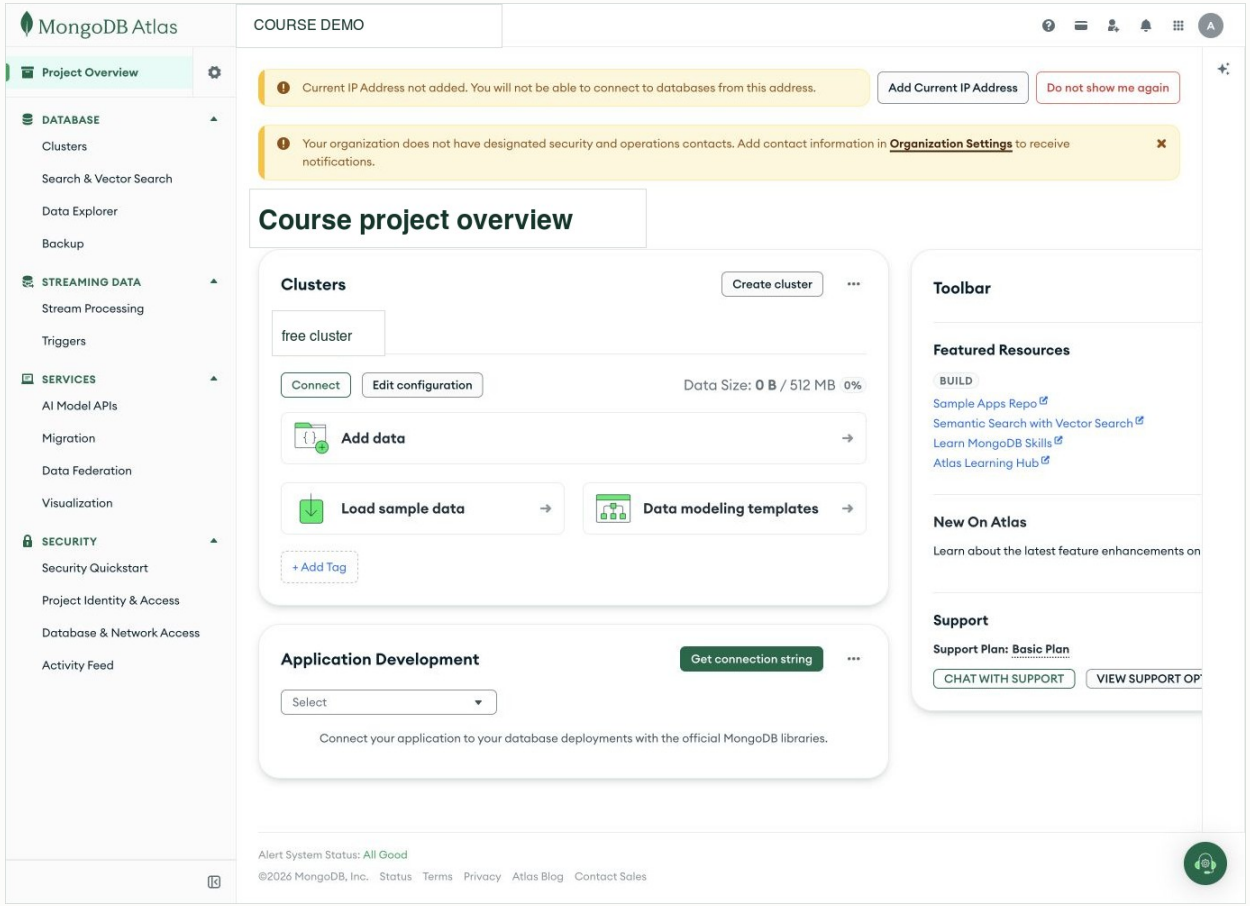
Run setup once. Repeat the selected import cell.
To try another target, clean up this run before starting another.

Dates and lists cross a model boundary

Source field	SQLite	MongoDB	PostgreSQL
cveID	TEXT primary key	Unique cveID index	text primary key
dateAdded	YYYY-MM-DD text	YYYY-MM-DD string	date value
cwes	JSON text	Array	jsonb array

The notebook chooses calendar dates, not times of day.
Verification converts SQL dates and JSON text back to the source form before comparing the selected fields.

Atlas needs the notebook runtime address



Connect > Drivers provides the Python URI.

Add the IPv4 /32 printed by the notebook. Your laptop can use a different address.

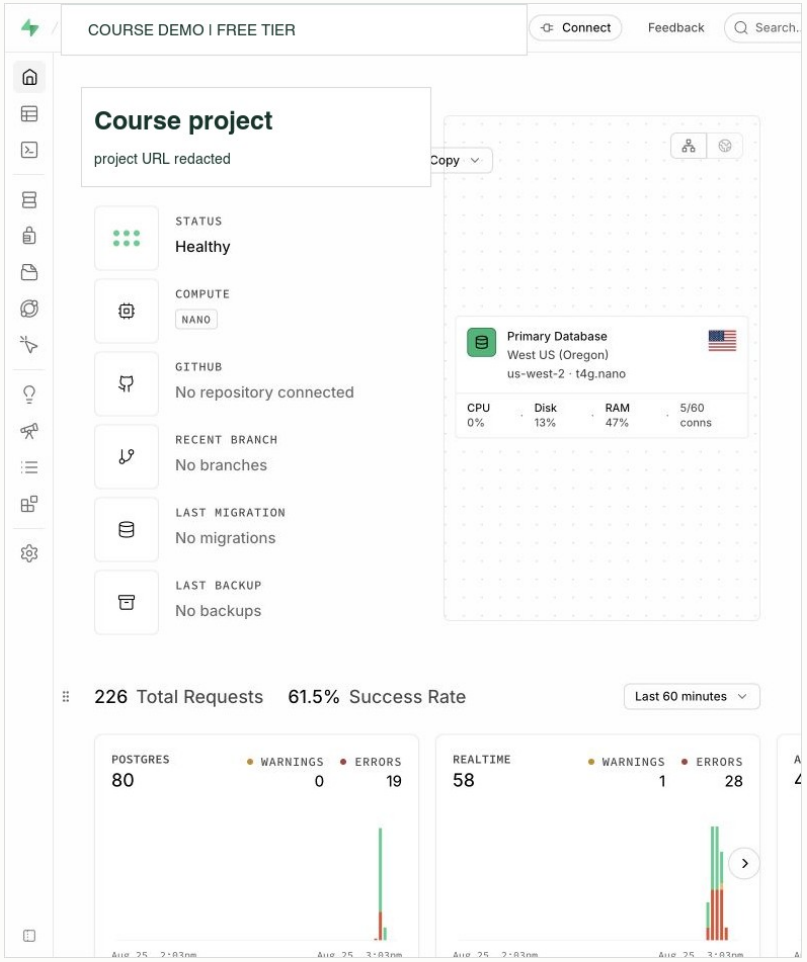
Redacted interface, August 25, 2026. Add Current IP refers to the browser address. The notebook asks you to confirm its own rule before connecting.

A bounded Atlas connection

```
atlas_client = MongoClient(
    atlas_uri, tls=True, tlsInsecure=False,
    server_api=ServerApi("1", strict=True),
    serverSelectionTimeoutMS=10000,
    timeoutMS=10000,
)
atlas_client.admin.command("ping")
```

Connection excerpt after the notebook's hidden URI prompt.
The complete cell clears the URI and closes a failed client.
A ping does not load or verify a dataset.

The Supabase connection panel



Connect > Session pooler

Use the database password and exact pooler username.
The account login and API key serve different purposes.

Use sslmode=require for encryption.
verify-full also verifies identity
when configured with the provider CA.

Redacted overview, August 25, 2026. The connection dialog is not shown.
The shared session pooler supports IPv4 notebook networks.

A parameterized PostgreSQL read

```
with postgres_connection.transaction():
    with postgres_connection.cursor() as cursor:
        cursor.execute(sql.SQL("SET LOCAL search_path TO {}").format(
            sql.Identifier(postgres_schema)))
        cursor.execute(
            "SELECT cve_id, product FROM kev_sample WHERE cve_id = %s",
            (known_id,),
        )
        print(cursor.fetchone())
```

After the selected PostgreSQL import. known_id comes from the source.
The schema is an identifier. The CVE ID is a separately bound value.

A repeat import uses the existing target

Operation	Records before	Records after
First import of this snapshot	0	75
Same source, same target, second import	75	75
Reset setup before every import	0 each time	75 each time

Only the second row demonstrates repeating against existing output.
The notebook also compares values, because equal counts can hide changes.

An upsert has an explicit match

```
for record in records:
    atlas_collection.replace_one(
        {"cveID": record["cveID"]},
        record,
        upsert=True,
    )
```

Match an existing CVE ID and replace its selected fields.
Insert only when that ID is absent. A unique index enforces the key.

This import does not delete missing source records or preserve earlier versions.

Failure can leave different intermediate states

Failure during the fourth write	What happens to the first three
SQL writes inside one transaction	An error rolls back that transaction's earlier writes
Separate MongoDB replace_one calls	Completed document writes remain
Retry the notebook's unchanged source	Stable IDs allow the remaining import to complete

Atomicity defines which changes succeed together.
Idempotency defines the effect of repeating the operation.

A matching count can conceal the wrong title

After changing one imported vulnerability title	Outcome
Record count is 75	Pass
Known CVE identifier exists	Pass
Vendor counts match	Pass
All eight fields of the known record match	Fail

The notebook checks the known record against its selected source.
Rerunning the unchanged source repairs that value.
One known record does not prove every untested field is correct.

Vendor groups and product groups answer different questions

Vendor label	Records
Microsoft	16
Cisco	7
"SimpleHelp "	3
Ubiquiti	3
Adobe	2

Product label	Records
Catalyst SD-WAN Manager	4
Windows	4
Defender	3
SimpleHelp	3
UniFi OS	3

Each row counts selected catalog entries with that exact field value.
Ties break by label. The quoted vendor label retains a trailing space.

A column name and a value have different roles

```
GROUP_FIELD = "product"
allowed_columns = {
    "vendorProject": "vendor_project", "product": "product"
}
group_column = allowed_columns[GROUP_FIELD]
sql.Identifier(group_column)
```

The mapping restricts which column may enter a SQL command.
The PostgreSQL path quotes that name as an identifier.
A %s value placeholder cannot substitute for a column name.

Lab, Part 2: import and write the handoff

Choose one target and repeat its import cell.
Change GROUP_FIELD from vendorProject to product.
Compare the displayed grouping and known record with the source.

Write a short handoff for a future maintainer: your question,
two key measurements, the repeat-run result, and a limitation.

Submit one completed notebook in Brightspace, individually.
No separate report, screenshot, matrix, or repository is required.

Cleanup removes the practice objects

Target	Notebook cleanup	Separate action
SQLite	Close this in-memory database	None
Atlas	Drop only this run's kev_sample collection	Remove the temporary runtime /32 rule
PostgreSQL	Drop kev_sample; drop its schema only if empty	Review any provider-side access changes

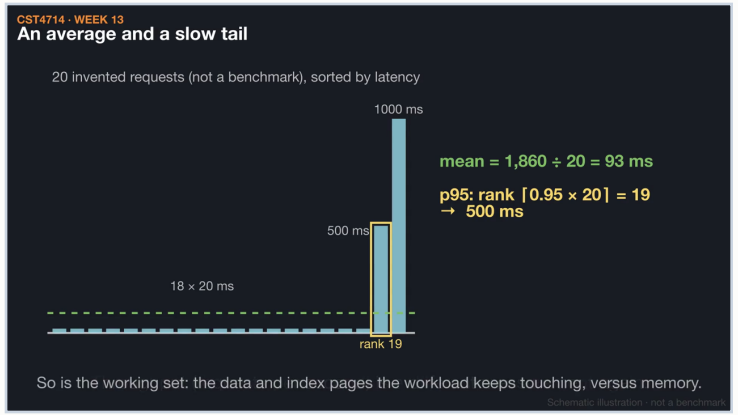
Other tables and collections remain. Closing a client does not pause a hosted database or remove an Atlas network rule.

Explaining the work and planning the next change

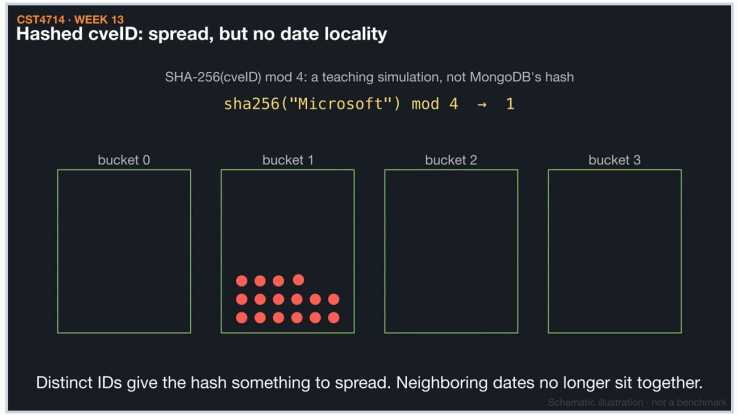
I built a small, repeatable import from a documented public source.
I compared distribution keys, tested the same import twice,
and checked both a query result and stored record values.
I can explain which behavior I ran and which scaling claims
this small dataset cannot support.

For your project, discuss one source, one important query, and a measured reason to change capacity. Use the existing final-project assignment.
Next: diagnosing disagreements when an application uses multiple stores.

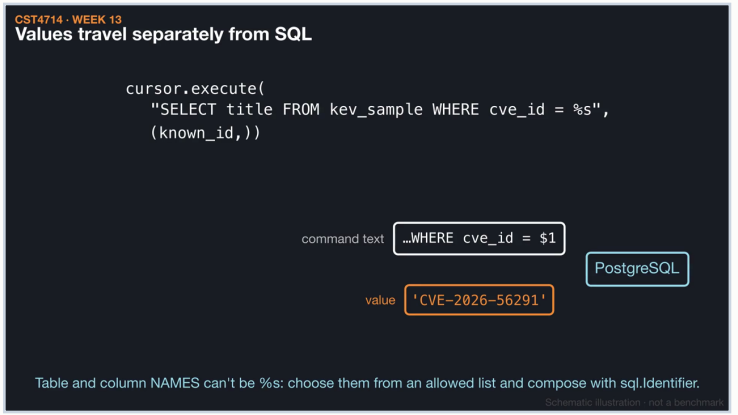
This week's videos



Capacity: measure before you scale (1:17)



Shard keys: distribution and targeting (1:37)



A repeatable Python import (1:28)

Click a thumbnail to play on GitHub. Silent, with captions and .srt subtitles; diagrams are schematic.